

# Statement

We both worked together on each exercise. On another date Timur made the screenshots and pdf for E3 while Tony did it for E4.

For E3 we used Chatgpt firstly to verify our solutions and occasionally for the following uses:

1. Find orcid of Alexander Schulthies via this generated query:

```
PREFIX dblp: <https://dblp.org/rdf/schema#>
```

```
SELECT DISTINCT ?person ?name ?orcid
WHERE {
  ?person a dblp:Person ;
    dblp:creatorName ?name ;
    dblp:orcid ?orcid .

  FILTER(CONTAINS(LCASE(STR(?name)), "alexander schultheis"))
}
```

so we can use it for E3 f)

2. For h) we had this query which got us the false solution so we asked Chatgpt where the problem was. It suggested using the operator "SAMPLE" and "DISTINCT" in the "HAVING" segment and explained it to us.

Our originally query was:

```
PREFIX dblp: <https://dblp.org/rdf/schema#>
```

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
```

```
SELECT ?author ?name ?affiliation (COUNT(DISTINCT ?publication) AS ?publicationCount)
WHERE {
```

```
  ?author rdf:type dblp:Person ;
```

```
    dblp:creatorName ?name ;
```

```
    dblp:affiliation ?affiliation .
```

```
  FILTER (CONTAINS(?affiliation,"Germany"))
```

```
  ?publication dblp:authoredBy ?author .
```

```
}
```

```
GROUP BY ?author ?name ?affiliation
```

```
HAVING (?publicationCount >= 800)
```

```
ORDER BY DESC(?publicationCount)
```

```
LIMIT 10
```

3. In the doc of DBLP we couldn't find the Property to get the firstAuthor of a Publication so we asked the KI how to and got this answer:

```
?signature dblp:signatureOrdinal 1 ;
```

```
    dblp:signatureCreator ?firstAuthor .
```

?firstAuthor dblp:creatorName ?firstAuthorName .  
so we can use it for E3 i9 and j)

4. For our version of i) we got the error:

Assertion `singleResult.size() == 1` failed. An expression returned a vector expression result that contained an unexpected amount of entries.. Please report this to the developers. In file  
"/srv/qllever/qllever/src/engine/GroupByImpl.cpp " at line 442

to find out why, we asked KI so we can understand and fix that. Chatgpt said something like the following:

The problem was that our originally query nested aggregation and fallback logic, e.g. `COALESCE(SAMPLE(...), "unknown")`, which caused an internal QLever/SPARQL engine error. We fixed it by first calculating `SAMPLE(...)` in a subquery and then applying `COALESCE(...)` in the outer query. We used AI to understand the error message and restructure the query accordingly.

For E4 we did the tutorial from the assignment sheet. We further looked in the literature and found a similar example from the Protégé paper ([https://protege.stanford.edu/conference/2006/submissions/slides/OWLTutorial\\_Part1.pdf](https://protege.stanford.edu/conference/2006/submissions/slides/OWLTutorial_Part1.pdf)), namely the Pizza ontology example (<https://protege.stanford.edu/ontologies/pizza/pizza.owl>).

We used this example as best practice and designed our model analogous to it, as the pizza maker has similarities to the sandwich maker exercise.

We validated our model using the reasoner HermiT 1.4.3.456 and the OntoGraf graph view. This can be seen in the screenshots and in the reasoner log file with the inferred class hierarchy picture.